AI ENGINEER TRACK · DAY 1 · ≈ 2 HOURS

LLM Fundamentals that drive cost, latency & reliability

Tokens · Context windows · Sampling · Rate limits | Project: FinDoc RAG Assistant — Day 1 scaffold

GOAL

Day 1 goal: Understand the four levers that control every LLM call in production — tokens, context, sampling, rate limits — and build an instrumented LLM client that logs tokens, latency and cost per call.

Why it matters: Every design decision later (chunk size, RAG vs long-context, model routing, caching) is a cost/latency/quality trade-off measured in tokens. Interviewers at 50+ LPA open with *"estimate the cost of this system"* or *"why is it slow?"* — today gives you the numbers to answer.

1 Concepts — only what production uses

A. Tokens

- LLMs read and write **tokens** (sub-word pieces from a BPE tokenizer), not characters or words. Billing, context limits and rate limits are all in tokens.
- Rule of thumb for English prose: **~4 characters ≈ 1 token**, ~0.75 words per token. **Numbers, tables, code and Indic scripts tokenize worse** (more tokens per character) — financial statements cost noticeably more per page than narrative text.
- Tokenizers are **model-specific**. OpenAI GPT-4o / GPT-4.1 family uses `o200k_base` (count with `tiktoken`). Claude, Llama, Mistral use their own — use the provider's count API or the `usage` field returned in each response as the source of truth.
- Every response returns `usage.prompt_tokens` and `usage.completion_tokens`. Log both — this is your cost and capacity telemetry.

B. Context window

- Context window = **input tokens + output tokens** in one call. A 128K window does not mean 128K of input — reserve room for `max_tokens` of output.
- Models also have a separate **max output** limit (often 4K–32K), far smaller than the window.
- Long context ≠ free accuracy: recall drops for facts buried in the **middle** of very long prompts, and cost/TTFT grow with every input token. This is why RAG (send 5 relevant chunks, not 300 pages) wins on cost, latency and grounding.
- **Prompt caching:** providers discount repeated prompt *prefixes* (OpenAI/Azure automatically above a size threshold; Anthropic via explicit cache markers). Put static content (system prompt, tool schemas, few-shot) **first**, variable content (user query, retrieved chunks) **last**.

C. Sampling parameters

Parameter	What it does	Production setting
temperature	Scales randomness of next-token choice. 0 = most likely token.	0–0.2 for RAG answers, extraction, classification, tool calls. 0.7–1.0 only for creative/brainstorm.
top_p	Samples only from the smallest set of tokens whose probability sums to p.	Leave default; tune **either** temperature **or** top_p, never both.
max_tokens	Hard cap on output length.	Always set it (cost + latency guard). Check `finish_reason` — `"length"` means truncated.
stop	Sequences that end generation early.	Useful for templated outputs; less needed with JSON mode.

Parameter	What it does	Production setting
seed	Best-effort reproducibility.	Helps tests; not a guarantee of identical outputs.
reasoning effort	On reasoning models (o-series, GPT-5-class, Claude extended thinking): controls hidden thinking tokens.	Low/medium by default; thinking tokens are billed as output.

Note: many reasoning models ignore or reject `temperature`/`top_p`. Your client must not assume every model accepts every parameter.

D. Latency — the two numbers that matter

- **TTFT (time to first token):** queueing + processing the input. Grows with input length and provider load.
- **TPS (output tokens/sec):** generation speed. Output is produced one token at a time, so **total latency ≈ TTFT + output_tokens ÷ TPS**. Output length is usually the biggest lever.
- Levers: **stream** (perceived latency = TTFT), cap output length, smaller model for simple steps, prompt caching, parallelise independent calls (async), fewer sequential LLM hops in agents.

E. Cost

COST FORMULA

Cost per call = (input_tokens × input_price + output_tokens × output_price) ÷ 1,000,000

Prices are quoted **per 1M tokens**. Output is typically **3–5× pricier** than input; cached input is discounted. Small models are often 10–20× cheaper than flagship ones. Always pull current rates from the provider's pricing page into config — never hard-code them in logic.

Back-of-envelope (interview style, example rates $2.50 in / $10 out per 1M): 5,000 queries/day × (4,000 input + 400 output tokens) → input 20M × 2.50 = $50, output 2M × 10 = $20 → **$70/day ≈ $2,100/month**. Move 70% of traffic to a model 15× cheaper and the bill drops by roughly two-thirds.

F. Rate limits & capacity

- Limits are **TPM** (tokens/min) and **RPM** (requests/min) per deployment/model. Exceeding them returns **HTTP 429**.
- Handle with **exponential backoff + jitter**, honour the `retry-after` header, and cap retries.
- **Azure OpenAI** (what most GCCs use): *Standard / pay-as-you-go* deployments share capacity — latency varies. *Provisioned Throughput (PTU)* reserves capacity for predictable latency at a fixed cost. Scale beyond one deployment's quota by load-balancing across deployments/regions (e.g., via API Management).

Your DE edge: treat an LLM call exactly like a pipeline stage — structured logs per call (tokens, latency, cost, model, status), retries with backoff, idempotency, and metrics you can aggregate. Most candidates can call an API; few can tell you p95 latency and cost per query. Today you build that.

2 Hands-on (≈ 60 min) — FinDoc RAG Assistant: Day 1 scaffold

This is the running project for the whole track: a cited Q&A assistant over annual reports, broker PDFs and earnings-call transcripts for your 8 Defence/IT companies. Today: repo + an instrumented LLM client. Every later day plugs into this client.

Step 1 · Repo + environment (10 min)

```
findoc-rag/
├── .env                      # keys — never commit
├── config/models.yaml        # model names + prices per 1M tokens
├── data/raw/                 # 1 annual report PDF to start (e.g. HAL or Infosys)
├── logs/llm_calls.jsonl      # one line per LLM call
├── src/llm/client.py         # instrumented client  ← today
└── scripts/day01_tokens.py   # token + cost profiling ← today

python -m venv .venv && source .venv/bin/activate
pip install openai tiktoken pypdf python-dotenv pyyaml tenacity
touch scripts/__init__.py src/__init__.py src/llm/__init__.py   # make packages importable
# .gitignore  →  .env   logs/   .venv/
# Run every command from INSIDE findoc-rag/ (paths below are relative to it)
```

Step 2 · Config (5 min)

```
# config/models.yaml  — fill prices from the provider pricing page today
default_model: gpt-4o-mini
models:
  gpt-4o-mini: {input_per_1m: 0.0, output_per_1m: 0.0}   # <-- update
  gpt-4o:      {input_per_1m: 0.0, output_per_1m: 0.0}   # <-- update

# .env  — put your real key inside the quotes; add .env to .gitignore
OPENAI_API_KEY="sk-proj-..."
LLM_PROVIDER=openai
```

Step 3 · Instrumented client (25 min) — type it, don't paste blindly

```
# src/llm/client.py
import json, os, time, uuid, yaml
from pathlib import Path
from dotenv import load_dotenv
from openai import OpenAI, RateLimitError, APITimeoutError, APIConnectionError
from tenacity import retry, stop_after_attempt, wait_random_exponential, retry_if_exception_type

load_dotenv()
CFG = yaml.safe_load(Path("config/models.yaml").read_text())
LOG = Path("logs/llm_calls.jsonl"); LOG.parent.mkdir(exist_ok=True)

CLIENT = OpenAI(max_retries=0)   # reads OPENAI_API_KEY; tenacity owns retries (no double retry)

def _cost(model, p_tok, c_tok):
    price = CFG["models"].get(model, {"input_per_1m": 0, "output_per_1m": 0})
    return (p_tok * price["input_per_1m"] + c_tok * price["output_per_1m"]) / 1_000_000

def _log(rec):
    with LOG.open("a") as f: f.write(json.dumps(rec) + "\n")

RETRYABLE = (RateLimitError, APITimeoutError, APIConnectionError)

@retry(retry=retry_if_exception_type(RETRYABLE),
       wait=wait_random_exponential(min=1, max=30), stop=stop_after_attempt(5))
def chat(messages, model=None, temperature=0.0, max_tokens=512, stream=False, tag=""):
    model = model or CFG["default_model"]
    # initialise ALL locals (lowercase!) - finish is read inside the stream loop
    t0 = time.perf_counter(); ttft = None; text = []; usage = None; finish = None
    kwargs = dict(model=model, messages=messages, temperature=temperature, max_tokens=max_tokens)
    if stream:
        resp = CLIENT.chat.completions.create(**kwargs, stream=True,
                                              stream_options={"include_usage": True})
        for chunk in resp:
            if chunk.choices:
                delta = chunk.choices[0].delta.content
                if delta:
                    if ttft is None: ttft = time.perf_counter() - t0
                    text.append(delta); print(delta, end="", flush=True)
                finish = chunk.choices[0].finish_reason or finish
            if chunk.usage: usage = chunk.usage      # final chunk carries usage
        print()
    else:
        resp = CLIENT.chat.completions.create(**kwargs)
        text.append(resp.choices[0].message.content or "")
        usage, finish = resp.usage, resp.choices[0].finish_reason
    total = time.perf_counter() - t0
    p, c = (usage.prompt_tokens, usage.completion_tokens) if usage else (0, 0)
    rec = {"id": str(uuid.uuid4()), "ts": time.time(), "tag": tag, "model": model,
           "stream": stream, "temperature": temperature,
           "prompt_tokens": p, "completion_tokens": c,
           "ttft_s": round(ttft, 3) if ttft else None, "total_s": round(total, 3),
           "tps": round(c / (total - (ttft or 0)), 1) if c and total > (ttft or 0) else None,
           "finish_reason": finish, "cost_usd": round(_cost(model, p, c), 6)}
    _log(rec)          # note: log metrics, NOT raw prompts (PII/confidential data)
    if finish == "length":
        print(f"[warn] output truncated at max_tokens={max_tokens}")
    return "".join(text), rec
```

Step 4 · Profile a real annual report (20 min)

```
# scripts/day01_tokens.py
# run from inside findoc-rag/:  python -m scripts.day01_tokens data/raw/HAL_AR-2025-26.pdf
import sys, json, statistics, tiktoken
from pypdf import PdfReader
from src.llm.client import chat, _cost

enc = tiktoken.get_encoding("o200k_base")
reader = PdfReader(sys.argv[1])
pages = [p.extract_text() or "" for p in reader.pages]

# 1) Token profile of the document
counts = [len(enc.encode(t)) for t in pages]
ratios = [len(t) / max(1, n) for t, n in zip(pages, counts) if n > 50]
print(f"pages={len(pages)} total_tokens={sum(counts):,} "
      f"avg chars/token={statistics.mean(ratios):.2f}")
# skip near-blank pages (covers, back pages) so we pick real tables
worst = sorted([i for i in range(len(pages)) if counts[i] > 200],
               key=lambda i: len(pages[i]) / counts[i])[:3]
print("lowest chars/token pages (likely tables):", [w + 1 for w in worst])
print("cost to send WHOLE report as input once: $",
      round(_cost("gpt-4o-mini", sum(counts), 0), 4), "(mini) vs $",
      round(_cost("gpt-4o", sum(counts), 0), 4), "(4o)")

# 2) Sampling: same question, 3 runs each at temp 0 and 1.0
ctx = pages[worst[0]][:6000]
q = [{"role": "system", "content": "Answer only from the context. Be concise."},
     {"role": "user", "content": f"Context:\n{ctx}\n\nWhat are the 3 key figures on this page?"}]
for t in (0.0, 1.0):
    outs = [chat(q, temperature=t, max_tokens=150, tag=f"temp{t}")[0] for _ in range(3)]
    print(f"\ntemp={t}: distinct answers = {len(set(outs))}/3")

# 3) Streaming vs non-streaming latency
_, a = chat(q, max_tokens=300, stream=False, tag="nostream")
_, b = chat(q, max_tokens=300, stream=True, tag="stream")
print(json.dumps({"non_stream_total": a["total_s"], "stream_ttft": b["ttft_s"],
                  "stream_total": b["total_s"], "tps": b["tps"]}, indent=2))
```

DELIVERABLE

Record these 5 numbers in your notes (you will quote them in interviews):

- Total tokens of one annual report, and chars/token on a prose page vs a table page.
- Cost to send the whole report once on a small vs a large model.
- How many distinct answers at temperature 0 vs 1.0.
- TTFT vs total latency with streaming; output tokens/sec.
- A `finish_reason="length"` case — rerun with `max_tokens=20` to see it.

3 Top interview questions — crisp model answers

Q1. Our context window is 128K. Why not put the whole annual report in the prompt instead of building RAG?

Three reasons. **Cost:** a 300-page report is ~150–250K tokens; paying that on every query is 30–50× the cost of sending the top-5 relevant chunks. **Latency:** TTFT grows with input size. **Accuracy:** recall degrades for facts in the middle of long contexts, and you lose citations. Long context is useful for one-off whole-document tasks (summarise once, then cache); RAG is the default for repeated Q&A over a corpus.

Q2. What temperature do you use for a RAG answer or data extraction, and why?

0–0.2. These are fidelity tasks — the goal is the most supported answer, not variety. Higher temperature raises the chance of picking a lower-probability, unsupported token, which shows up as hallucinated figures. I tune temperature or top_p, not both, and on reasoning models I control reasoning effort instead.

Q3. We set temperature = 0 but outputs still differ between runs. Why, and how do you get repeatability?

Temperature 0 is near-greedy, but GPU floating-point non-determinism, dynamic batching and model updates behind the same name can change the top token when probabilities are close. For repeatability: pin a dated model version, set seed, use structured output with schema validation so variance can't break consumers, and cache responses for identical inputs. Evaluate on distributions, not single runs.

Q4. Users say the assistant is slow. How do you diagnose and fix it?

Split latency into **TTFT** and **generation time** (output tokens ÷ TPS) from per-call traces. High TTFT → trim or cache the prompt (static prefix first), fewer retrieved chunks, check queueing/429 retries, or move to PTU. Long generation → cap output, ask for concise format, smaller model for simple steps. Always stream to the UI. In agents, cut sequential LLM hops and run independent calls in parallel.

Q5. Estimate the monthly LLM cost for an internal assistant used by 2,000 analysts.

State assumptions: ~10 queries/user/day → 20K/day; ~4K input (system + 5 chunks + question) and ~400 output tokens. That's 80M input + 8M output tokens/day. Multiply by the per-1M prices; output usually dominates per token, input dominates in volume for RAG. Then name levers: route easy queries to a small model, prompt caching on the static prefix, response cache for repeat questions, cap output — typically a 50–70% reduction.

Q6. You're getting 429s from Azure OpenAI in production. What do you do?

Short term: exponential backoff with jitter honouring retry-after, bounded retries, and a queue so bursts are smoothed. Then capacity: check TPM vs actual token usage (max_tokens reservation counts against quota), raise quota, spread load across deployments/regions behind a gateway, and move latency-critical traffic to PTU. Reduce demand with caching and smaller models for simple calls.

Q7. Why are output tokens more expensive and slower than input tokens?

Input tokens are processed in parallel in one forward pass. Output tokens are generated one at a time, each requiring a forward pass over the growing sequence — more compute per token and inherently sequential. That's why limiting output length is the strongest latency and cost lever.

4 Common mistakes & production gotchas

Mistake	What happens	Do this instead
Not checking `finish_reason`	Truncated answers / broken JSON reach users silently	Treat `"length"` as an error: retry with higher cap or shorter prompt
Budgeting only input tokens	Cost and latency estimates off by 2–5×	Model input and output separately; output is pricier and slower
Counting tokens with the wrong tokenizer	Context overflow errors on long docs	Use the model's tokenizer; trust the usage field
Filling the context window to the limit	Request rejected or output cut short	Reserve max_tokens + a safety margin
Variable text at the start of the prompt	Prompt cache never hits	Static system prompt/tools first, query and chunks last
Logging full prompts/responses	PII and confidential financials in logs	Log metrics + IDs; store payloads only in a secured, masked store

Mistake	What happens	Do this instead
Retrying every error	Duplicate side-effects, wasted spend	Retry only 429 / timeout / 5xx; never 400s; make tool actions idempotent
Setting both temperature and top_p	Unpredictable behaviour, hard to debug	Tune one; leave the other at default
Using an unpinned model alias	Behaviour changes after a silent model update	Pin dated model versions / Azure deployment versions

5 Done-checklist — tick each item before moving on

- [ ] I can explain what a token is and why tables/numbers cost more tokens than prose.
- [ ] I can state the cost formula and do a back-of-envelope monthly estimate out loud in under 2 minutes.
- [ ] I can explain context window vs max output, and three reasons RAG beats stuffing a long context.
- [ ] I know which temperature to use for extraction/RAG vs creative tasks, and why temp 0 isn't fully deterministic.
- [ ] I can split latency into TTFT and generation time and name 4 levers to reduce each.
- [ ] I can explain 429s, TPM/RPM, backoff with jitter, and Standard vs PTU on Azure OpenAI.
- [ ] Repo `findoc-rag` created; `src/llm/client.py` runs with streaming and non-streaming.
- [ ] `logs/llm_calls.jsonl` has entries with tokens, TTFT, total latency, TPS, finish_reason, cost.
- [ ] `day01_tokens.py` profiled one annual report; I've written down my 5 numbers.
- [ ] I triggered and handled a `finish_reason="length"` case.
- [ ] Answered all 7 interview questions aloud without reading the answers.